MoverMate: архитектура кода

Сервис «переезд без хаоса»: план задач, учёт коробок и вещей, QR-этикетки, поиск, контроль погрузки и распаковки.

Хакатон PROD, кейс №1, команда 20. Документ описывает то, что реально написано и запущено на 16 октября 2026. Что не сделано, отмечено отдельно (раздел 11).

38	8	67 + 47	98%
эндпоинтов API	таблиц в базе	тестов сервера и сайта	покрытие сервера

1. Что делает система и какие у неё ограничения

Пользователь создаёт проект переезда (название, дата, адреса необязательны), получает готовый план задач на три этапа (до, в день, после), заводит коробки с комнатой назначения, быстро вписывает содержимое, печатает этикетки с QR-кодом, отмечает погрузку, доставку и распаковку и ищет вещи по названию. Если вещь потерялась, система подсказывает, в каких нераспакованных коробках её искать.

Требование ТЗ	Как выполнено
Простота запуска, без внешних сервисов	Один Docker-образ, SQLite в томе; QR строится в браузере, внешних API нет
Номера коробок уникальны, повторный скан не создаёт дубль	Ограничение БД `UNIQUE(project_id, number)`, `GET` карточки ничего не создаёт
Статусы только по допустимым переходам	Таблица переходов в сервисе, ответ 409 с перечнем допустимых
Одинаковые запросы дают одинаковые результаты	Поиск и списки с вторичными ключами сортировки (номер коробки, id)
Данные переживают обновление страницы	Данные в БД, список проектов в `localStorage`
Персональные данные необязательны, нет скрытых передач	Нет аккаунтов; адреса и ответственный необязательны; нет внешних вызовов
Сотни коробок и тысячи вещей, пагинация	Индексы, `page` / `page_size`, нагрузочный проект на 300 коробок и 3000 вещей
Журнал событий и проверка работоспособности	Таблица `events`, `GET /api/health` с запросом к БД

2. Общая схема

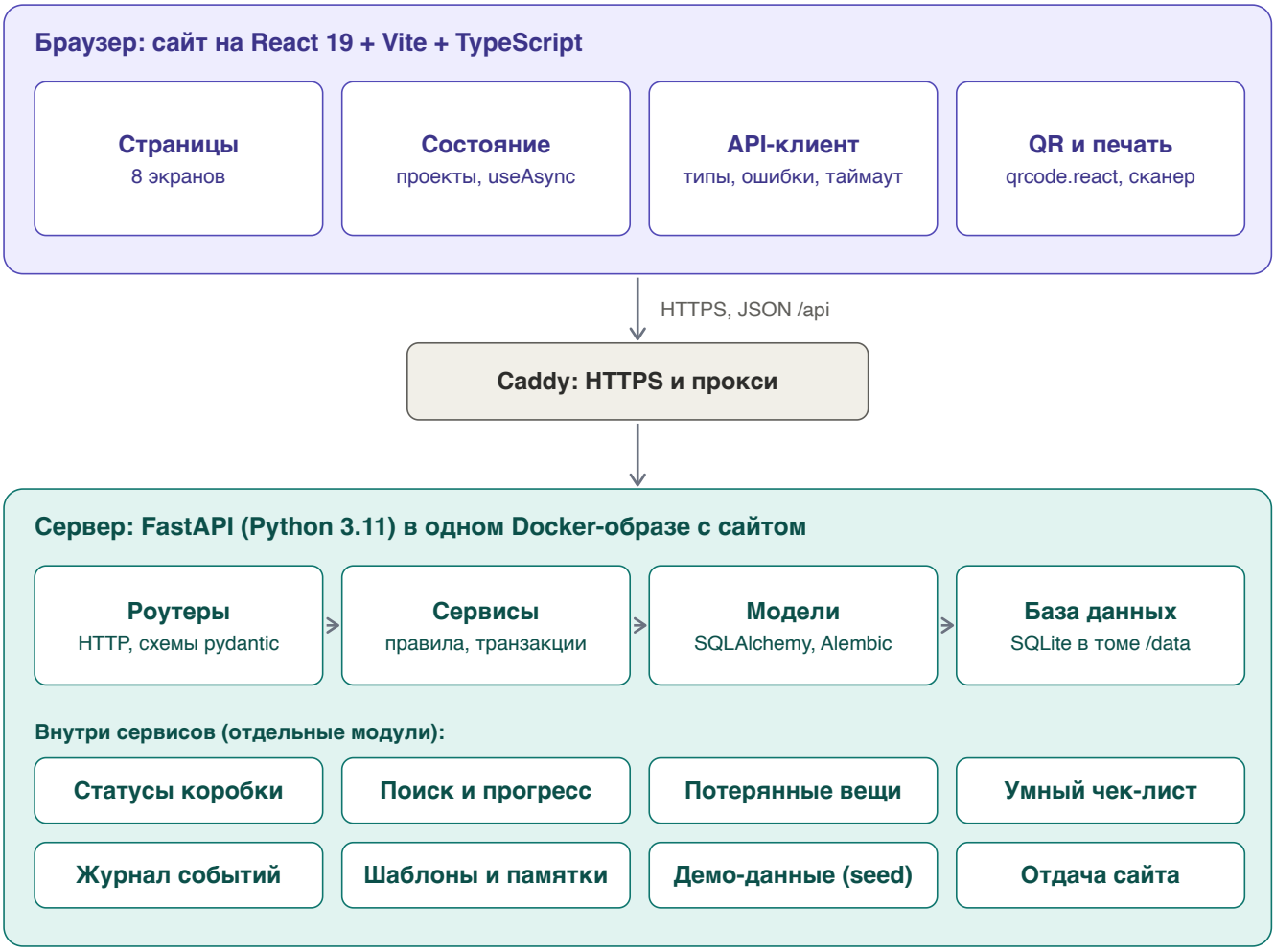


Рис. 1. Слои системы. Сайт и сервер едут в одном образе, база лежит в томе.

Сайт и API работают на одном origin: Caddy отдаёт сайт и проксирует `/api` на FastAPI, поэтому CORS не нужен. В режиме разработки запросы `/api` проксирует Vite (порт 5173 на порт 8000).

3. Сервер (backend)

Три слоя с чёткими обязанностями: роутер принимает и проверяет запрос, сервис содержит все правила и работает с транзакцией, модель описывает таблицы. Бизнес-правил в роутерах нет, поэтому сервисы можно тестировать без HTTP.

Модуль	Что внутри
`main.py`	Создание приложения, прослойка журнала запросов (request_id, время), обработчики ошибок, подключение роутеров, запуск справочников при старте
`routers/`	`projects` (проекты, задачи, события), `boxes` (коробки, вещи, поиск, этикетки, прогресс, первый день, потерянные, напоминания), `assistant`, `reference` (шаблоны, памятки), `health`, общий `confirm_param` для удаления
`services/boxes.py`	Самый крупный модуль (433 строки): выдача номера, машина статусов, массовые переходы, вещи, поиск и ранжирование, этикетки, прогресс

`services/extras.py`	Чек-лист первого дня, потерянные вещи и подсказки, напоминания
`services/tasks.py`, `projects.py`	Задачи с сортировкой по этапу и сроку; создание проекта с шаблонами
`services/assistant.py`	Умный чек-лист на локальных правилах, идемпотентное добавление
`services/events.py`	Запись события в той же транзакции, что и изменение
`models.py, db.py`	8 таблиц SQLAlchemy; движок с включёнными внешними ключами для SQLite
`schemas.py`	Контракт API на pydantic: ограничения длины, обрезка пробелов, пустая строка как «не заполнено»
`reference_data.py`, `seed.py`	Шаблоны задач, памятки, чек-лист первого дня, правила умного чек-листа; демо-проект и нагрузочный проект
`static_site.py`	Отдаёт собранный сайт и маршруты SPA (включая ссылки из QR), защищён от `../`; `/api/…` без маршрута остаётся честным 404

Пример: смена статуса коробки



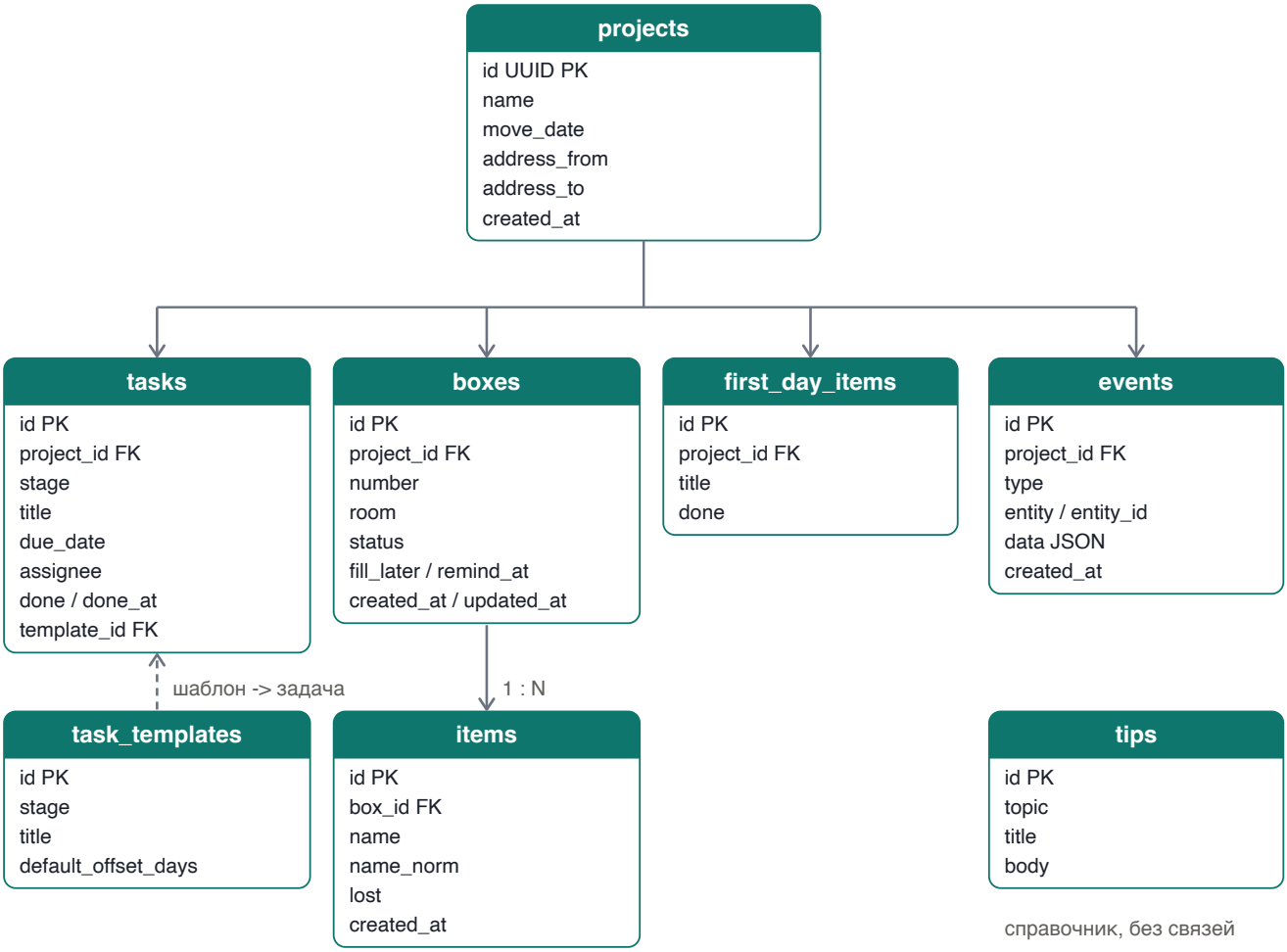
Рис. 2. Один запрос сквозь слои. Блокировка и запись события в одной транзакции.

Единый формат ошибок

```
{"detail": "Нельзя перевести коробку из «packing» в «unpacked». Допустимо: packed",
 "code": "conflict", "request_id": "8a3232f8d2b9"}
```

Коды: 404 нет сущности, 409 недопустимый переход, 422 неверные данные или нет `confirm=true` при удалении (с перечнем полей в `errors`), 500 внутренняя ошибка (стек в логе, клиенту только `request_id`). Логи в формате JSON с идентификатором запроса.

4. Модель данных



Все дочерние таблицы удаляются каскадом вместе с проектом (ON DELETE CASCADE).

Ограничения: UNIQUE(project_id, number) у boxes; UNIQUE(project_id, template_id) у tasks (шаблон не дублируется).

Рис. 3. Таблицы и связи. Миграция: `backend/alembic/versions/0001_initial_schema.py`.

- **Номер коробки** хранится отдельным числом, а не как `id`. Сервер выдаёт `max(number)+1` внутри проекта под блокировкой строки проекта; при гонке запись повторяется до трёх раз, а `UNIQUE` не даёт создать дубль. Удалённые номера не переиспользуются: напечатанная этикетка не должна указывать на чужую коробку.
- `name_norm` в вещах: имя без регистра, `ё = е`, схлопнутые пробелы. Поиск не зависит от функций СУБД и одинаково работает на SQLite и PostgreSQL.
- **Комната** это строка в коробке: у комнаты нет своих атрибутов, прогресс по комнатам считается группировкой.
- **Индексы:** `boxes(project_id, room, status)`, `items(box_id)`, `items(name_norm)`, `tasks(project_id, stage)`, `events(project_id, id)`.
- **Справочники** (`task_templates`, `tips`) сидятся при каждом старте и добавляют только недостающее.

5. Доменная логика: статусы коробки



Рис. 4. Машина статусов. Таблица переходов: `TRANSITIONS` в `services/boxes.py`.

Проверка переходов вынесена в чистую функцию `check_transition` и покрыта тестом на все 25 пар статусов. Смена статуса, запись в журнал и проверки идут в одной транзакции со `SELECT … FOR UPDATE` по коробке: две одновременные отметки идут по очереди. Массовая погрузка комнаты (`bulk-transitions`) атомарна: один недопустимый номер отклоняет всю операцию.

Правила, на которых держится сервис

Область	Правило
Поиск	Совпадения ранжируются: точное, с начала слова, вхождение; затем номер коробки и id. `%` и `_` в запросе обычные символы
Прогресс	«Погружено» в комнате = погружена + доставлена + распакована; «Распаковано» = распакованные / все коробки комнаты; охват описью = коробки с вещами / все
Напоминания	Коробка «заполнить позже», без вещей, срок не задан или наступил; вычисляются при запросе, без планировщика
Подсказки о потерянной вещи	Только нераспакованные коробки: где записана (1,0), где лежит похожее по основам слов (0,8), та же комната (0,6); не больше 10, порядок стабилен
Умный чек-лист	Правила для флагов «дети», «животные», «другой город» в `reference_data.py`; добавляются только отсутствующие по названию, срок считается от даты переезда
Удаление	Всегда с `confirm=true`, иначе 422: случайный запрос не удаляет данные

6. API

Все 38 эндпоинтов описаны в Swagger (`/api/docs`) и в `docs/API.md`. Доступ без авторизации: проект идентифицируется неугадываемым UUID в пути, ссылка на проект работает как секретный адрес. Длинные списки идут страницами (`page`, `page_size` до 200).

Метод	Путь	Назначение
Служебные (3)		
GET	`/api/health`	Проверка работоспособности с запросом к БД
GET	`/api/task-templates`	Шаблоны задач
GET	`/api/tips`	Памятки (фильтр topic)
Проекты (5)		
POST	`/api/projects`	Создать проект: шаблонные задачи и чек-лист первого дня
GET	`/api/projects/{pid}`	Получить проект
PATCH	`/api/projects/{pid}`	Изменить название, дату, адреса
DELETE	`/api/projects/{pid}?confirm=true`	Удалить проект со всем содержимым
GET	`/api/projects/{pid}/events`	Журнал событий (страницы, фильтр type)
Задачи (5)		
GET	`/api/projects/{pid}/tasks`	Задачи (фильтры stage, done)
POST	`/api/projects/{pid}/tasks`	Добавить свою задачу
PATCH	`/api/projects/{pid}/tasks/{tid}`	Изменить или отметить выполнение
DELETE	`/api/projects/{pid}/tasks/{tid}?confirm=true`	Удалить задачу
POST	`/api/projects/{pid}/tasks/apply-templates`	Добавить недостающие шаблонные задачи
Коробки (9)		
GET	`/api/projects/{pid}/boxes`	Список коробок (room, status, fill_later, number, страницы)
POST	`/api/projects/{pid}/boxes`	Создать коробку, номер выдаёт сервер
GET	`/api/projects/{pid}/boxes/{n}`	Карточка коробки (её открывает QR)
PATCH	`/api/projects/{pid}/boxes/{n}`	Комната, «заполнить позже»
DELETE	`/api/projects/{pid}/boxes/{n}?confirm=true`	Удалить коробку с вещами
POST	`/api/projects/{pid}/boxes/{n}/transitions`	Сменить статус по допустимым переходам
POST	`/api/projects/{pid}/boxes/bulk-transitions`	Массовая смена статуса, всё или ничего
GET	`/api/projects/{pid}/boxes/{n}/label`	Данные этикетки (qr_path)
GET	`/api/projects/{pid}/labels`	Пакет этикеток (numbers=1,2,3, до 100)
Вещи (7)		

GET	/api/projects/{pid}/boxes/{n}/items	Вещи в коробке (страницы)
POST	/api/projects/{pid}/boxes/{n}/items	Добавить вещь
POST	/api/projects/{pid}/boxes/{n}/items/bulk	Быстрый ввод: строка = вещь, до 200
PATCH	/api/projects/{pid}/items/{id}	Переименовать, пометить потерянной
DELETE	/api/projects/{pid}/items/{id}?confirm=true	Удалить вещь
GET	/api/projects/{pid}/items/lost	Потерянные вещи
GET	/api/projects/{pid}/items/{id}/hints	Где искать потерянную вещь
Поиск и контроль (3)		
GET	/api/projects/{pid}/search?q=	Поиск вещи среди всех коробок
GET	/api/projects/{pid}/progress	Статусы и прогресс по комнатам
GET	/api/projects/{pid}/reminders	Коробки, которые пора описать
Первый день (4)		
GET	/api/projects/{pid}/first-day	Чек-лист первого дня
POST	/api/projects/{pid}/first-day	Добавить пункт
PATCH	/api/projects/{pid}/first-day/{id}	Отметить или переименовать
DELETE	/api/projects/{pid}/first-day/{id}?confirm=true	Удалить пункт
Умный чек-лист (2)		
POST	/api/projects/{pid}/assistant/checklist	Предложить задачи (дети, животные, другой город)
POST	/api/projects/{pid}/assistant/checklist/apply	Добавить предложенное, идемпотентно

7. Сайт (frontend)

React 19, TypeScript, Vite, `react-router-dom`, обычный CSS без UI-библиотек. Зависимостей минимум, код короткий. Сканер QR (`html5-qrcode`) подгружается отдельным куском только при открытии экрана.

Маршруты и экраны

Путь	Экран
`/`	План: этапы, задачи, коробка первого дня, умный чек-лист
`/boxes`	Коробки: поиск вещи и номера, комнаты, напоминания
`/projects/:pid/boxes/:n`	Карточка (сюда ведёт QR)
… `/label`	Этикетка: номер, комната, QR

Слои кода

Папка	Роль
`pages/`	Экраны, по одному файлу
`components/`	Каркас (`Shell`), окна (`Modal`), этикетка, формы
`state/`	Проекты в браузере, `useAsync`, `useDebounced`
`api/`	Типы по контракту, клиент, функции эндпоинтов
`ui/`	Форматы, `StateView`, бейдж статуса

`/labels`, `/labels/print`	Список и печать всех этикеток
`/progress`	Счётчики, комнаты, потерянные вещи
`/scan`, `/tips`, `/new`	Сканер, памятки, новый проект

Как работают экраны

- Каждый экран грузит данные через `useAsync` и показывает единые состояния «загрузка / ошибка с повтором / пусто» (`StateView`). Действия пользователя ловят ошибки сервера и показывают понятный текст.
- Клиент API (`api/client.ts`) ходит на `/api` того же origin, ставит таймаут 20 секунд, разбирает формат ошибок сервера и превращает ответ nginx в HTML (502) в понятное сообщение.
- Выбранный проект хранится в `localStorage` (с защитой от недоступного хранилища). Если открыли ссылку из QR на проект, которого нет в списке, он добавляется автоматически.
- **QR:** этикетка кодирует обычную ссылку `{origin}/projects/{pid}/boxes/{n}`; QR рисуется в браузере (`qrcode.react`). Любая камера телефона открывает её; на сайте есть и свой сканер, который принимает ссылку с любым адресом сайта.
- **Печать:** `window.print()` и правила `@media print` оставляют только этикетку; системный диалог печатает или сохраняет в PDF.
- **Адаптивность:** на телефоне вкладки снизу, от 900 пикселей слева; светлая и тёмная темы; цвет каждого статуса один на всех экранах.

8. Ключевые решения

Решение	Почему
Сайт вместо мобильного приложения	Симулятор и сборка APK съедали время при сроке в три дня; сайт проще проверять, а QR открывается любой камерой.
Без авторизации, доступ по UUID проекта	ТЗ не требует аккаунтов и просит минимум персональных данных. Минус: ссылку нельзя отозвать
Три слоя: роутер, сервис, модель	Правила не размазаны по контроллерам; сервисы тестируются без HTTP
Таблица переходов статусов	Новый статус или правило это одна строка плюс тест
Откат статуса на один шаг назад	Помощники ошибаются при отметках; пропуск шагов остаётся запрещённым (требование ТЗ)
Номер в транзакции под блокировкой + UNIQUE	Уникальность гарантирует база, даже если сервис ошибётся
Один образ с сайтом и API	Проще деплой и шаблон CI организаторов (`Dockerfile` в корне); SQLite в томе: мало движущихся частей
Caddy с автоматическим HTTPS	Камера сканера работает только по HTTPS; сертификат получается и продлевается сам
Умный чек-лист на правилах, без внешнего ИИ	ТЗ запрещает скрытые передачи данных; результат предсказуем и проверяем тестами

9. Развёртывание

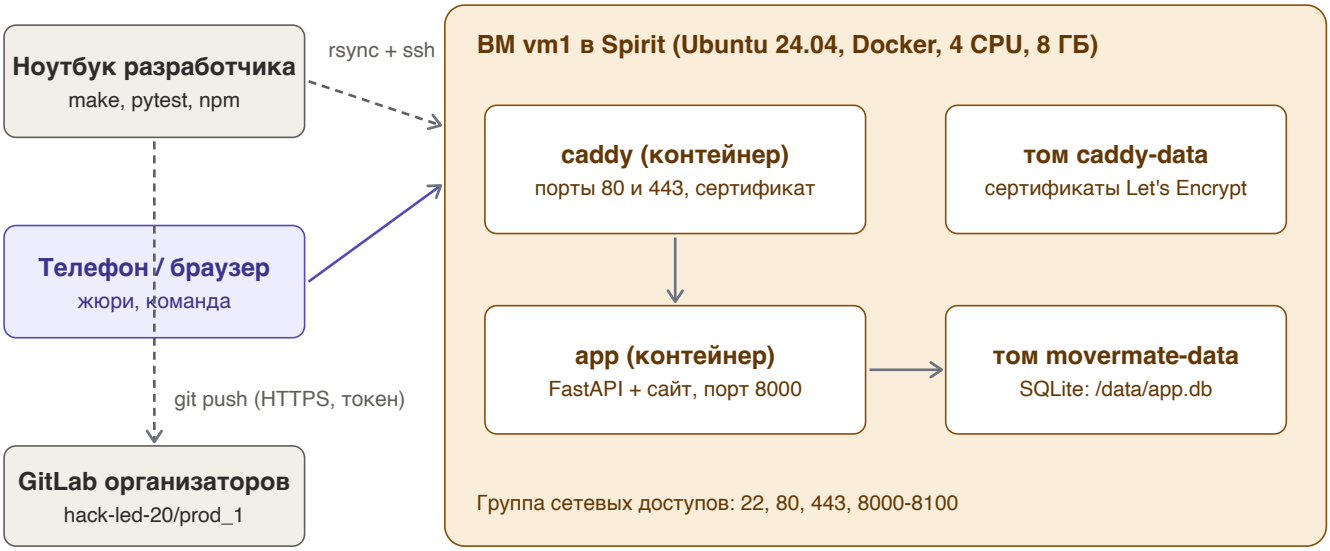


Рис. 5. Окружение. Сейчас деплой выполнен вручную, из пайплайна GitLab ещё нет.

- **Образ:** `Dockerfile` в корне, два этапа: сборка сайта (Node 22) и сервер (Python 3.11). При старте контейнер применяет миграции и запускает `uvicorn`; проверка живости `/api/health`.
- **Compose:** `deploy/docker-compose.yml` поднимает `app` и `caddy`; имя сервера для HTTPS задаётся в `deploy/.env` (`SITE_ADDRESS`).
- **Адрес:** https://hack-led-20-vm1-12902f.vmaas-spirit.t-education.com (сертификат Let's Encrypt).
- **Передеплой:** `rsync` исходников на ВМ и `docker compose up -d --build` (подробно в `docs/DEPLOY_VM.md`); проверка `scripts/preflight.sh <адрес>`.
- **Локально:** `scripts/run-site.sh` (сервер отдаёт API и сайт на порту 8000) или `make dev` + `npm run dev`.

10. Тестирование и проверка

Что	Сколько	Что проверяется
Сервер (pytest)	67 тестов, покрытие 98%	Правила статусов (все пары), номера и дубли, поиск и порядок, прогресс, подсказки, напоминания, массовая погрузка, чек-лист и его идемпотентность, миграция совпадает с моделями, отдача сайта и защита от `../`
Сайт (vitest)	47 тестов	Разбор ссылки из QR, форматы и склонения, клиент API (сеть, 409/422, 502), хранилище проектов (в том числе недоступное), состояния экрана, форма проекта
Сквозная проверка	`scripts/preflight.sh`	Сайт и ссылка из QR, health, Swagger, демо-данные, полный сценарий ТЗ на временном проекте; зелёный на локальном стеке и на ВМ

Пропущен один тест: параллельная выдача номеров нужна PostgreSQL (`TEST_DATABASE_URL`). Нагрузка (300 коробок, 3000 вещей) замерялась вручную: списки, поиск и прогресс отвечают за единицы миллисекунд.

11. Что не сделано и ограничения

- **Пайплайн GitLab.** Деплой пока вручную, а критерий жюри требует сборку и деплой из CI. В шаблоне организаторов (`templates/l2sh-template`) нужны: `.gitlab-ci.yml` с шагами build, test, deploy и переменные `SSH_PRIVATE_KEY`, `DEPLOY_VM`; сервис-аккаунт для образов в тенанте уже создан.
- **Код в GitLab** организаторов ещё не отправлен (ждёт токен доступа).
- **Не реализованы** план упаковки комнаты и опись коробки по фото (описаны в `docs/API.md`).
- **Доступ по ссылке нельзя отозвать;** при утечке ссылки данные проекта открыты. Персональные данные в системе необязательны.
- **База на одной ВМ** (SQLite в томе), резервных копий нет. Параллельные запросы на PostgreSQL проверены только тестом, который в обычном прогоне пропускается.
- **Камера в браузере** требует HTTPS: на ВМ он включён, на локальном запуске работает только на `localhost`. Саму камеру на телефоне ещё не проверяли.

12. Структура репозитория

```
backend/            сервер: app/ (код), alembic/ (миграции), tests/ (67 тестов)
frontend/           сайт: src/ (код), Dockerfile и nginx.conf (запуск без Caddy)
deploy/             docker-compose.yml, Caddyfile, .env.example для ВМ
Dockerfile          образ: сайт + сервер
docs/               CASE.md (минимум Т3), API.md, ARCHITECTURE.md, TEAM_PLAN.md, DEPLOY_VM.md
scripts/            preflight.sh, run-site.sh, push-to-gitlab.sh, gitlab-login.sh
```